Class 3 - SQL Notes

WHERE, IN, BETWEEN, LIKE, ORDER BY, LIMIT, UPDATE, Transactions, Aggregates + Practice Tasks

How to read: green text starting with -- is a comment (ignored by SQL, written for humans). Dark text is the actual SQL. Run everything top to bottom in PostgreSQL, after the Class 2 setup.

1. Filtering Rows with WHERE

Until now SELECT returned every row. WHERE keeps only the rows that match a condition. These queries run on the customers, products and orders tables created in the previous class (the setup script is not repeated here).

```
-- WHERE <condition> is written AFTER FROM. SQL checks the condition for every row
-- and keeps only the rows where it is TRUE.
-- > means "greater than". Other comparison operators:
--   =  equal        <>  or  !=  not equal       >  greater       <  less
--   >= greater or equal                         <= less or equal
-- Result: Laptop (55000), Headphones (2500), Office Chair (8000) -> 3 rows.
SELECT * FROM products WHERE price > 2000;
```

2. OR, IN and NOT IN

Use OR when ANY one of several conditions is enough. IN is a shorter way to write many ORs on the same column.

```
-- OR: a row is kept if the left condition OR the right condition is true.
-- Text values go in single quotes and are CASE-SENSITIVE: 'Delhi' is not 'delhi'.
-- Result: Priya, Rohit, Vikram -> 3 rows.
SELECT * FROM customers WHERE city = 'Delhi' OR city = 'Noida';

-- IN ( ... ) = "is equal to any value in this list". Same result as the OR query above,
-- but shorter and easier to read when the list grows.
SELECT * FROM customers WHERE city IN ('Delhi' , 'Noida');

-- NOT IN ( ... ) = "is none of the values in this list".
-- Result: Aarav, Sneha, Ananya, Isha -> 4 rows.
-- IMPORTANT TRAP: Karan Malhotra (city = NULL) does NOT appear, even though NULL is
-- not Delhi or Noida. NULL means "unknown", so the comparison is neither true nor false
-- and the row is dropped. To include NULL rows you must add:  OR city IS NULL
SELECT * FROM customers WHERE city NOT IN ('Delhi' , 'Noida');
```

3. BETWEEN with AND

BETWEEN selects a range. AND joins two conditions: BOTH must be true for the row to be kept.

```
-- A query can be written on several lines; SQL only ends at the semicolon.
SELECT * FROM orders
-- BETWEEN a AND b is INCLUSIVE: it includes a and b themselves.
--   so '2025-03-01' and '2025-05-31' are both part of the range.
-- It is the same as:  order_date >= '2025-03-01' AND order_date <= '2025-05-31'
-- AND status = 'Delivered' adds a second rule: both rules must be true.
-- Result: 9 rows (the Cancelled and Returned orders are removed by the status rule).
WHERE order_date BETWEEN '2025-03-01' AND '2025-05-31'  AND status = 'Delivered';
```

4. Mixing AND with OR (use brackets)

AND is evaluated before OR, exactly like multiplication before addition in maths. Brackets force the order you actually want.

```
SELECT * FROM orders
-- Read it as:  status must be 'Delivered'  AND  ( at least ONE of the two inside the
-- bracket must be true ).
--   quantity >= 3               -> bulk orders
--   order_date >= '2025-03-01'  -> orders placed on or after 1 March 2025
-- The bracket is calculated first, then the result is combined with AND.
-- Result: 11 rows.
WHERE status = 'Delivered' AND ( quantity >= 3 OR order_date >= '2025-03-01' );

-- WITHOUT the brackets:  status = 'Delivered' AND quantity >= 3 OR order_date >= '2025-03-01'
-- SQL reads it as:  (Delivered AND qty >= 3)  OR  (date >= 2025-03-01)
-- so Cancelled and Returned orders placed after 1 March would sneak in. Always use brackets
-- when AND and OR appear together.
```

5. Pattern Search with LIKE and ILIKE

LIKE searches text by pattern instead of an exact match. % is a wildcard that stands for any number of characters (including zero). ILIKE is the PostgreSQL version that ignores upper/lower case.

```
-- 'A%'  = starts with A, followed by anything. LIKE is CASE-SENSITIVE.
-- Result: Aarav Mehta, Ananya Gupta -> 2 rows.
SELECT * FROM customers WHERE name LIKE 'A%'; -- starts with A

-- '%A' = anything, then ends with a capital A.
-- Result: 0 rows, because every name ends with a small letter (LIKE respects case).
SELECT * FROM customers WHERE name LIKE '%A'; -- ends with A

-- ILIKE = case-INsensitive LIKE (works only in PostgreSQL). 'a' and 'A' both match.
-- Result: Aarav, Priya, Rohit, Ananya, Karan -> 5 rows.
SELECT * FROM customers WHERE name ILIKE '%A'; -- ends with A / a

-- '%A%' = A anywhere in the text (start, middle or end) = "contains".
-- Result: all 8 customers, because every name has an a somewhere.
SELECT * FROM customers WHERE name ILIKE '%A%'; -- contains a / A

-- Same idea with k. Result: Sneha Kapoor, Vikram Singh, Karan Malhotra -> 3 rows.
SELECT * FROM customers WHERE name ILIKE '%k%'; -- contains k / K
```

6. Sorting with ORDER BY

Without ORDER BY the database may return rows in any order. ORDER BY sorts the result. It always comes AFTER WHERE and is the last part of a normal query (before LIMIT).

```
-- ORDER BY <column>: default direction is ASC (ascending, smallest first).
-- Result: SQL Book 550, Python Book 650, Desk Lamp 1200 ... Laptop 55000.
SELECT product_name, price FROM products ORDER BY price; -- ascending

-- DESC = descending (largest first). Result: Laptop 55000 comes first.
SELECT product_name, price FROM products ORDER BY price DESC; -- descending
```

7. Sorting by Many Columns and NULLS LAST

You can sort by more than one column. The second column is used only to break ties in the first.

```
SELECT name, city FROM customers
-- 1st sort key: city A to Z.
-- NULLS LAST = rows where city is NULL go to the very end.
--   (PostgreSQL puts NULLs FIRST when sorting DESC and LAST when sorting ASC by default;
--    NULLS FIRST / NULLS LAST lets you choose explicitly.)
-- 2nd sort key: name Z to A. It matters only when two customers share the same city,
--   e.g. Gurugram: Sneha Kapoor is listed before Aarav Mehta.
-- Each column can have its own direction, separated by commas.
ORDER BY city ASC NULLS LAST , name DESC; -- sort by city, then name, blank at the end
```

8. LIMIT and OFFSET (Top-N and Pagination)

LIMIT restricts how many rows come back. OFFSET skips rows first. Always combine them with ORDER BY, otherwise 'top' has no meaning.

```
-- top 2 most expensive products

SELECT product_name, price FROM products
-- Sort so the most expensive product is first.
ORDER BY price DESC
-- LIMIT 2 = return only the first 2 rows of the sorted result.
-- Result: Laptop 55000, Office Chair 8000.
LIMIT 2;

-- SKIP 1 AND top 2 most expensive products
-- (the original note says "skip 2" but the query uses OFFSET 1, which skips ONE row)

SELECT product_name, price FROM products
ORDER BY price DESC
-- OFFSET 1 = throw away the first 1 row (Laptop), then LIMIT 2 takes the next 2.
-- Result: Office Chair 8000, Headphones 2500.
-- Pagination formula:  OFFSET = (page_number - 1) * rows_per_page
LIMIT 2 OFFSET 1;
```

9. Changing Data with UPDATE

UPDATE modifies existing rows. The WHERE part decides WHICH rows change. The pattern SELECT, UPDATE, SELECT lets you see the before and after.

```
-- Step 1 - BEFORE: look at the row first. SQL Book currently costs 550.
SELECT * FROM products WHERE product_name = 'SQL Book';

-- UPDATE <table> SET <column> = <new value> WHERE <which rows>;
-- SET gives the new value. WHERE limits the change to the SQL Book row only.
-- DANGER: if you forget WHERE, EVERY row in the table is updated!
-- Several columns can be changed together:  SET price = 599, category = 'Books'
UPDATE  products SET price = 599 WHERE product_name = 'SQL Book';
-- Step 2 - AFTER: check that the price is now 599.
SELECT * FROM products WHERE product_name = 'SQL Book';

-- Put the original price back so the practice data stays the same.
UPDATE  products SET price = 550 WHERE product_name = 'SQL Book';
-- Confirm it is 550 again.
SELECT * FROM products WHERE product_name = 'SQL Book';
```

10. Transactions: BEGIN, ROLLBACK, COMMIT

A transaction groups several statements into one all-or-nothing unit. Until you COMMIT, the changes are only temporary and can be undone with ROLLBACK. This is how banks make sure a transfer never leaves money half-moved.

```
-- BEGIN starts a transaction. Everything below is a draft until COMMIT or ROLLBACK.
BEGIN;
    -- Deletes every Delivered order, but only inside this transaction (a trial run).
    DELETE FROM orders WHERE status = 'Delivered'; -- temp delete
    -- COUNT(*) counts the rows still left. Inside the transaction we see 2 rows
    -- (the Cancelled and the Returned order); the delete looks real to us.
    SELECT COUNT(*) FROM orders; -- check remaining row count
-- ROLLBACK cancels the whole transaction: all 13 deleted orders come back.
ROLLBACK; -- undo everything since BEGIN
```

```sql
-- Second transaction, this time we keep the change.
BEGIN;
    -- Removes the single Returned order. Still only a draft at this point.
    DELETE FROM orders WHERE status = 'Returned'; -- temp delete
    -- Shows 14 rows (15 minus the one deleted).
    SELECT COUNT(*) FROM orders; -- check remaining row count
-- COMMIT saves the changes PERMANENTLY. After this you cannot ROLLBACK.
-- (The original comment here says "undo everything since BEGIN" - that was copied from
--  the ROLLBACK line by mistake. The correct meaning is: make everything since BEGIN permanent.)
COMMIT;


-- Count only Delivered orders. Result: 13 (nothing was lost from the first transaction).
SELECT COUNT(*) FROM orders WHERE status = 'Delivered';

-- Final check: 14 rows. The Returned order is gone for good because of the COMMIT.
SELECT * FROM orders;
```

11. Aggregate Functions (COUNT, SUM, AVG, MIN, MAX)

An aggregate function reads MANY rows and returns ONE value. Used without GROUP BY, the whole table is summarised into a single result row.

```sql
SELECT
    -- COUNT(*) counts every row, whatever it contains. 14 orders.
    COUNT(*)                        AS total_orders,
    -- COUNT(DISTINCT col) counts each different value once, so repeat buyers count once.
    -- Buyers are customers 1,2,3,4,5,6,8 -> 7.
    COUNT(DISTINCT customer_id)     AS unique_buyers,
    -- SUM adds up all the values in the column: total units sold = 32.
    SUM(quantity)                   AS units_sold,
    -- AVG = SUM / number of rows = 32 / 14 = 2.2857...
    -- ROUND(value, 2) trims it to 2 decimals -> 2.29.
    ROUND(AVG(quantity) , 2)        AS avg_qty,
    -- MIN gives the smallest value; for dates this is the earliest date: 2025-01-25.
    MIN(order_date)                 AS first_order,
    -- MAX gives the largest value; for dates the latest date: 2026-01-19.
    MAX(order_date)                 AS last_order
-- All of the above are calculated from the orders table and return exactly ONE row.
-- AS gives each result column a readable name (alias).
FROM orders;

-- Good to know:
--   * COUNT(column) skips NULL values, COUNT(*) does not.
--   * SUM, AVG, MIN and MAX also ignore NULL values.
--   * An aggregate can be combined with WHERE: the WHERE filters the rows FIRST,
--     then the aggregate is calculated on what is left.
```

Practice Tasks (new tables, same topics)

Use the NEW tables below (employees and sales). Run the setup script once, then write every query yourself before checking the output. Solutions are intentionally not included. Add a comment above each answer explaining what your query does.

Setup script - run this first

```
DROP TABLE IF EXISTS sales, employees CASCADE;

CREATE TABLE employees(
    emp_id     SERIAL PRIMARY KEY,
    emp_name   VARCHAR(50) NOT NULL,
    dept       VARCHAR(20),
    city       VARCHAR(30),
    salary     NUMERIC(10, 2),
    bonus      NUMERIC(10, 2),                  -- NULL = no bonus
    hire_date  DATE
);

CREATE TABLE sales(
    sale_id    SERIAL PRIMARY KEY,
    emp_id     INT REFERENCES employees(emp_id),
    item       VARCHAR(30),
    qty        INT,
    sale_date  DATE,
    status     VARCHAR(15)                      -- 'Closed', 'Pending', 'Cancelled'
);

INSERT INTO employees (emp_name, dept, city, salary, bonus, hire_date) VALUES
('Rahul Khanna', 'Sales',   'Gurugram',   45000,  5000, '2022-04-11'),
('Neha Joshi',   'HR',      'Delhi',      38000,  NULL, '2021-08-02'),
('Amit Arora',   'IT',      'Noida',      72000,  8000, '2020-01-15'),
('Pooja Nair',   'IT',      'Gurugram',   68000,  NULL, '2023-03-20'),
('Sanjay Kumar', 'Sales',   'Delhi',      41000,  3000, '2023-07-01'),
('Kavya Iyer',   'Finance', 'Bengaluru',  55000,  4000, '2019-11-30'),
('Deepak Yadav', 'Sales',   NULL,         36000,  NULL, '2024-02-14'),
('Meera Kapoor', 'Finance', 'Delhi',      61000,  6000, '2021-05-25'),
('Arjun Bhatia', 'IT',      'Bengaluru',  80000, 10000, '2018-09-09'),
('Simran Kaur',  'HR',      'Noida',      34000,  NULL, '2024-06-03');

INSERT INTO sales (emp_id, item, qty, sale_date, status) VALUES
(1, 'Laptop',   2, '2025-01-12', 'Closed'),
(1, 'Mouse',    10,'2025-02-03', 'Closed'),
(5, 'Monitor',  3, '2025-02-20', 'Pending'),
(5, 'Keyboard', 5, '2025-03-08', 'Closed'),
(7, 'Laptop',   1, '2025-03-15', 'Cancelled'),
(7, 'Mouse',    4, '2025-04-01', 'Closed'),
(1, 'Monitor',  2, '2025-04-18', 'Closed'),
(5, 'Laptop',   3, '2025-05-02', 'Pending'),
(7, 'Keyboard', 6, '2025-05-21', 'Closed'),
(1, 'Laptop',   1, '2025-06-09', 'Cancelled'),
(5, 'Mouse',    8, '2025-06-25', 'Closed'),
(7, 'Monitor',  2, '2025-07-14', 'Closed');
```

A. WHERE and Comparison Operators

Task 1. Show all employees whose salary is greater than 50000.

Task 2. Show all employees who work in the IT department.

Task 3. Show all employees hired after 1 January 2022.

Task 4. Show all employees whose department is NOT Sales (use the not-equal operator).

B. OR, IN and NOT IN

Task 5. Show employees living in Delhi or Noida using OR.

Task 6. Write the same query again using IN.

Task 7. Show employees whose department is not Sales and not HR using NOT IN.

Task 8. Show employees whose city is NOT IN ('Delhi', 'Noida'). Deepak (city NULL) will be missing: explain why in a comment, then change the query so Deepak is included.

C. BETWEEN and AND

Task 9. Show employees whose salary is between 40000 and 65000 (both included).

Task 10. Show all sales made between 1 March 2025 and 30 June 2025 whose status is 'Closed'.

Task 11. Show employees hired during the year 2023 using BETWEEN.

D. AND with OR and Brackets

Task 12. Show employees who work in IT or Finance AND earn more than 60000, using brackets.

Task 13. Write the same query without brackets, run it, and explain in a comment why the result is different.

Task 14. Show Closed sales where qty is 5 or more OR the sale date is on or after 1 June 2025.

E. LIKE and ILIKE

Task 15. Show employees whose name starts with 'S'.

Task 16. Show employees whose name ends with a small 'a' using LIKE, then try a capital 'A' and note the result.

Task 17. Show employees whose name ends with 'a' or 'A' in one query using ILIKE.

Task 18. Show employees whose name contains 'an' (any case).

Task 19. Show all sales whose item starts with the letter 'M'.

F. ORDER BY and NULLS LAST

Task 20. Show emp_name and salary sorted by salary from lowest to highest.

Task 21. Show emp_name and salary sorted from highest to lowest.

Task 22. Sort employees by dept A to Z, and inside each department by salary highest first.

Task 23. Show emp_name and city sorted by city A to Z with NULL cities at the end, then by emp_name Z to A.

Task 24. Show emp_name and bonus sorted by bonus highest first with the NULL bonuses placed last.

G. LIMIT and OFFSET

Task 25. Show the 3 highest-paid employees.

Task 26. Skip the highest-paid employee and show the next 3.

Task 27. Show the 2 most recently hired employees.

Task 28. Write a query for 'page 2' of the employees list sorted by emp_name when each page shows 4 rows.

H. UPDATE

Task 29. Show Neha Joshi's row, increase her salary to 42000, show the row again, then change it back.

Task 30. Give every employee in the Sales department a bonus of 2000 (check the rows before and after).

WebiGeeks

Task 31. Change the status 'Pending' to 'Closed' only for sales with qty greater than 2. Check the result with a SELECT.

Task 32. In a comment, explain what would happen if you ran an UPDATE without a WHERE condition.

I. Transactions

Task 33. Start a transaction, delete all Cancelled sales, count the remaining rows, then ROLLBACK. Confirm with a count that all rows are back.

Task 34. Start a transaction, delete all Pending sales, count the remaining rows, then COMMIT. Confirm the rows are gone.

Task 35. In two comment lines, explain the difference between ROLLBACK and COMMIT.

J. Aggregate Functions

Task 36. Count the total number of rows in sales.

Task 37. Count how many different employees made at least one sale.

Task 38. Find the total qty sold across all sales.

Task 39. Find the average salary of all employees rounded to 2 decimals.

Task 40. Find the oldest and the newest hire_date in employees.

Task 41. Run COUNT(*) and COUNT(bonus) on employees. Explain in a comment why the two numbers differ.

Task 42. Write one single query that returns total sales, unique employees, total qty, average qty (2 decimals), first sale date and last sale date, each with a readable alias.

Task 43. Find the average salary of employees in the IT department only (aggregate with WHERE).

Task 44. Find the total qty of Closed sales made in 2025 only.

K. Mixed Challenge

Task 45. Show the top 3 Closed sales of 'Laptop' or 'Monitor' in 2025, sorted by qty highest first.

Task 46. Create your own table cars(car_id, model, brand, price, city, launch_date), insert 8 rows (include one NULL city), then write at least one query for each: WHERE with AND/OR and brackets, IN, BETWEEN, ILIKE, ORDER BY with NULLS LAST, LIMIT with OFFSET, UPDATE, a transaction with ROLLBACK, and one query that uses COUNT, SUM, AVG, MIN and MAX together.

Task 47. Add a comment above every line of your cars script explaining what it does.

WebiGeeks